

UNIVERSIDAD NACIONAL DE LA MATANZA

Departamento de Ingeniería e Investigaciones Tecnológicas

Visión Artificial

Trabajo Práctico N.º 4 · Informe de avance

Detección de personas y vehículos en imágenes aéreas para plataformas de drones

Comparación inicial YOLO11n / YOLO26n sobre VisDrone2019-DET y plan de benchmark en computadoras de a bordo (OBC) con y sin GPU

Integrantes	DNI
[Apellido, Nombre]	[00.000.000]
[Apellido, Nombre]	[00.000.000]
[Apellido, Nombre]	[00.000.000]
[Apellido, Nombre]	[00.000.000]

Docentes: [completar]

Destinatarios: equipo del proyecto (coordinación, equipo Jetson, equipo Raspberry Pi 5)

Ciclo lectivo 2026 · 2.º cuatrimestre

San Justo, Buenos Aires · 26 de septiembre de 2026

Resumen

El objetivo del TP4 es detectar diez categorías de personas y vehículos en imágenes captadas por drones, con un detector lo bastante liviano para ejecutarse a bordo de la aeronave. Sobre el conjunto VisDrone2019-DET se ajustaron dos detectores de tamaño *nano* preentrenados en COCO, YOLO11n y YOLO26n, con un protocolo idéntico: 50 épocas, entrada de 640 px, lote 4 y semilla 42, en una GPU RTX 5070 Laptop. En validación ambos modelos resultaron prácticamente equivalentes: mAP50–95 de 16,03 % y 15,85 %, mAP50 de 28,41 % y 28,45 %, y latencias de 26,6 y 26,7 ms por imagen, respectivamente. Las clases de objetos pequeños (*bicycle*, *people*, *awning-tricycle*) concentran los peores resultados. Como las diferencias son menores que la incertidumbre de una única ejecución, la pregunta relevante deja de ser qué modelo es más preciso en una GPU de escritorio. Lo que importa es cómo se comporta el detector en el hardware que efectivamente vuela.

Por ese motivo, el siguiente paso consiste en medir YOLO26n, original y optimizado, en dos computadoras de a bordo representativas: una NVIDIA Jetson (GPU integrada, TensorRT FP16) y una Raspberry Pi 5 (sólo CPU, NCNN). Se registrarán FPS sostenidos, latencia p50/p95, memoria, temperatura y precisión sobre una muestra fija de 300 imágenes de test-dev. El paquete de medición, las herramientas y las pruebas automatizadas ya están preparados y verificados localmente. Este informe describe el problema, los resultados obtenidos, el diseño del benchmark, el reparto de tareas y las decisiones que el equipo debe tomar para acercar el trabajo al problema operativo de un dron.

Palabras clave: detección de objetos, vehículos aéreos no tripulados, YOLO, VisDrone, computación embarcada, TensorRT, NCNN.

Síntesis para el equipo

Qué se hizo	Datos preparados y auditados; YOLO11n y YOLO26n entrenados 50 épocas y comparados; paquete de benchmark OBC generado, verificado y documentado (tp4/benchmark.py, BENCHMARK_OBC.md).
Qué se concluye	Precisión equivalente entre ambos modelos (diferencia de 0,18 p.p. en mAP50–95, con una sola semilla). Los resultados obtenidos en la laptop no permiten predecir el comportamiento en la placa.
Qué sigue	Exportar y medir YOLO26n en Jetson (TensorRT) y Raspberry Pi 5 (NCNN), comparar los resultados en conjunto y evaluar test-dev con el modelo fijado.
Qué decidir	Modelo exacto de Jetson y responsables; requisito operativo mínimo (FPS y latencia); incorporar o no YOLO11n como control en placa; medir o no consumo energético (sección 7.3).

Índice

Resumen	2
1. Introducción	4
1.1. Contexto: visión artificial embarcada en drones	4
1.2. Planteo del problema	4
1.3. Objetivos	4
1.4. Organización del informe	4
2. Datos: VisDrone2019-DET	5
3. Metodología de la comparación inicial	5
3.1. Modelos	5
3.2. Protocolo experimental	6
3.3. Métricas	6
4. Resultados de la comparación inicial	7
4.1. Métricas globales	7
4.2. Resultados por clase	7
4.3. Evolución durante el ajuste	8
4.4. Análisis cualitativo de errores	9
4.5. Discusión y elección del modelo para OBC	9
5. Del laboratorio al dron: requisitos operativos	10
5.1. Tamaño aparente de los objetos	10
5.2. Frecuencia de procesamiento	10
6. Siguiendo el paso: benchmark en computadoras de a bordo	11
6.1. Preguntas de investigación	11
6.2. Plataformas y variantes	11
6.3. Paquete común y reproducibilidad	11
6.4. Cadena medida	12
6.5. Protocolo de medición	12
6.6. Indicadores que se obtendrán	12
6.7. Flujo de trabajo	13
6.8. Estado de preparación	13
7. Plan de trabajo	13
7.1. Tareas, responsables y criterios de aceptación	13
7.2. Reglas para que los resultados sean comparables	14
7.3. Decisiones abiertas y propuestas	14
8. Limitaciones y riesgos	15
9. Conclusiones	15
Referencias	16
A. Anexo: trazabilidad de artefactos	16

1. Introducción

1.1. Contexto: visión artificial embarcada en drones

Los vehículos aéreos no tripulados (VANT, o UAV por su sigla en inglés) se emplean en vigilancia, búsqueda y rescate, monitoreo de tránsito, inspección de infraestructura y agricultura. En muchas de estas misiones, la percepción visual debe resolverse *a bordo*. Transmitir el video a una estación terrena para procesarlo añade latencia, depende de la calidad del enlace, consume ancho de banda y deja al sistema sin percepción si el enlace se pierde.

En la arquitectura habitual, un controlador de vuelo se encarga de la estabilización y la navegación. Junto a él, una *computadora de a bordo* (OBC, *On-Board Computer* o *companion computer*) ejecuta las tareas de mayor carga, como la detección de objetos. Esa computadora está sujeta a las restricciones SWaP (*Size, Weight and Power*): cada gramo y cada vatio reducen la autonomía de vuelo, la disipación térmica es limitada y no se dispone de una GPU de escritorio. En este trabajo se consideran dos familias representativas: placas con GPU integrada (NVIDIA Jetson) y placas sólo con CPU (Raspberry Pi 5).

1.2. Planteo del problema

La detección en imágenes aéreas es sensiblemente más difícil que en los conjuntos de referencia tomados desde el suelo, como COCO [3]:

- **Objetos muy pequeños.** Vistos desde decenas de metros, una persona o una bicicleta ocupan pocos píxeles. Al reducir la imagen al tamaño de entrada de la red, pueden quedar por debajo del límite que el modelo logra resolver.
- **Alta densidad.** VisDrone promedia unos 53 objetos anotados por imagen en entrenamiento y 71 en validación. Las escenas urbanas contienen cientos de instancias parcialmente ocluidas.
- **Clases visualmente próximas.** *Pedestrian* frente a *people*, *tricycle* frente a *awning-tricycle* y *car* frente a *van* se diferencian por detalles que se pierden al reducir la resolución.
- **Variabilidad de punto de vista.** Hay tomas cenitales y oblicuas, con distintas alturas, iluminaciones y escenarios.

A esas dificultades se agrega la restricción de cómputo: el detector debe funcionar en tiempo real en la OBC. Por lo tanto, la pregunta del proyecto no se reduce a *qué modelo es más preciso*. Interesa *qué combinación de modelo, motor de inferencia y placa ofrece el mejor compromiso entre precisión, velocidad, memoria y comportamiento térmico dentro de las restricciones de un dron*.

1.3. Objetivos

Objetivo general. Evaluar la viabilidad de un detector YOLO de tamaño *nano* para detectar personas y vehículos a bordo de un dron.

Objetivos específicos. La tabla 1 detalla los seis objetivos específicos e indica cuáles están cumplidos y cuáles pendientes a la fecha de este informe.

1.4. Organización del informe

La sección 2 describe el conjunto de datos y su preparación. Las secciones 3 y 4 presentan la metodología y los resultados de la comparación inicial. La sección 5 vincula esos resultados con los requisitos operativos de un dron. La sección 6 detalla el diseño del benchmark en OBC. La sección 7 organiza el trabajo pendiente y las decisiones abiertas. Por último, se enumeran las limitaciones y las conclusiones.

	Objetivo específico	Estado
O1	Preparar y auditar VisDrone2019-DET en formato YOLO, con trazabilidad (SHA-256).	Cumplido
O2	Ajustar YOLO11n y YOLO26n bajo un protocolo idéntico y compararlos en validación.	Cumplido
O3	Construir un paquete de benchmark reproducible y portable para placas OBC.	Cumplido
O4	Medir en Jetson y Raspberry Pi 5 el modelo original y su versión optimizada.	Pendiente
O5	Comparar las plataformas y discutir el compromiso precisión/velocidad/memoria.	Pendiente
O6	Evaluar test-dev completo con el modelo definitivo.	Pendiente

Tabla 1. Objetivos específicos del TP4 y estado al 26 de septiembre de 2026.



Figura 1. Revisión visual de cajas convertidas sobre tres imágenes de validación. La densidad y el tamaño reducido de peatones y motos ilustran la dificultad del dominio.

2. Datos: VisDrone2019-DET

VisDrone [1] es un conjunto de referencia de imágenes captadas por cámaras montadas en drones, en entornos urbanos y suburbanos de catorce ciudades de China. La tarea de detección (DET) anota diez categorías: *pedestrian*, *people*, *bicycle*, *car*, *van*, *truck*, *tricycle*, *awning-tricycle*, *bus* y *motor*. Se utilizaron los archivos distribuidos por Ultralytics [6], que replican los particionados oficiales: 6471 imágenes de entrenamiento, 548 de validación y 1610 de test-dev. Las imágenes tienen una resolución nativa mayor que la entrada de la red: en validación y test-dev predominan 1400×788 y 1360×765 píxeles, y hay tomas de hasta 1920×1080 .

La conversión al formato YOLO mapea las categorías originales 1–10 a los índices 0–9 y excluye las demás. Las regiones marcadas con *score* 0 (zonas ignoradas) no se usan como etiquetas. La auditoría encontró una única caja inválida, de ancho 4 y alto 0, que se excluyó y quedó registrada. Todos los archivos originales se conservan junto con su hash SHA-256.

Partición	Imágenes	Cajas convertidas	Cajas por imagen	Regiones score 0
Entrenamiento	6471	343 204	53,0	10 345
Validación	548	38 759	70,7	1 410
Test-dev	1610	75 102	46,6	2 445

Tabla 2. Conteos reales obtenidos por la auditoría de datos (`data/audit.json`).

Nota de uso. Los datos se utilizan con fines académicos, según las condiciones publicadas por los autores de VisDrone. No se redistribuyen imágenes ni pesos fuera del equipo.

3. Metodología de la comparación inicial

3.1. Modelos

Se compararon dos detectores de una etapa de la familia YOLO [2], ambos en su variante más pequeña (*nano*) y preentrenados en COCO:

- **YOLO11n** [4]: generación consolidada de Ultralytics. Su posprocesamiento requiere supresión de no máximos (NMS).
- **YOLO26n** [5]: generación más reciente. Según la documentación del fabricante, produce una salida *end-to-end* sin NMS, elimina el módulo DFL y está orientada explícitamente a dispositivos de borde. También declara mejoras en la función de pérdida pensadas para objetos pequeños.

En ambos casos se reemplazó la cabeza de 80 clases de COCO por una de 10 clases y se ajustó la red completa (*transfer learning*). Las clases de COCO no se comparan con las de VisDrone.

3.2. Protocolo experimental

Parámetro	Valor
Épocas / resolución / lote	50 / 640 × 640 / 4
Semilla / <i>workers</i>	42 / 0
Selección de pesos	<code>best.pt</code> según validación
Hardware	NVIDIA GeForce RTX 5070 Laptop (8 GB), Windows 11
Software	Python 3.14.4, PyTorch 2.13.0+cu130, Ultralytics 8.4.162
Ejecuciones	YOLO11n: 20260925T015117Z; YOLO26n: 20260925T231501Z

Tabla 3. Protocolo común a ambos modelos. Mismos particionados y versiones de software.

3.3. Métricas

Se informan *Precision*, *Recall*, mAP50 (precisión media por clase con umbral de IoU de 0,5, promediada entre clases) y mAP50–95 (promedio sobre umbrales de IoU de 0,50 a 0,95 en pasos de 0,05). El mAP50–95 exige mayor exactitud de localización y es el criterio principal de comparación. Todas las métricas provienen del evaluador de Ultralytics, **no** del evaluador oficial de VisDrone.

Además se midieron:

- **Latencia:** `predict` completo en la RTX 5070, 640 px, lote 1, con 5 calentamientos y 20 repeticiones sobre una misma imagen.
- **Auditoría visual de errores:** las 8 primeras imágenes de validación (871 objetos anotados), con confianza 0,25 e IoU 0,5, clasificando cada caja como verdadero positivo (TP), falso positivo (FP) o falso negativo (FN).

4. Resultados de la comparación inicial

4.1. Métricas globales

La tabla 4 resume la evaluación de cada modelo sobre el conjunto de validación completo, junto con la latencia y el costo de entrenamiento.

Métrica (validación)	YOLO11n	YOLO26n	Δ (26 – 11)
Precision	41,88 %	40,41 %	–1,47 p. p.
Recall	31,38 %	31,77 %	+0,39 p. p.
mAP50	28,41 %	28,45 %	+0,04 p. p.
mAP50–95	16,03 %	15,85 %	–0,18 p. p.
Latencia RTX 5070 (ms/imagen)	26,59	26,74	+0,15
Tiempo total de ajuste (h)	3,27	4,41	+34,9 %

Tabla 4. Evaluación de best.pt sobre validación (548 imágenes). Una ejecución por modelo; las diferencias no tienen significancia estadística establecida.

El rendimiento global es muy similar. YOLO11n muestra una ventaja descriptiva de 0,18 p. p. en mAP50–95 y YOLO26n una de 0,04 p. p. en mAP50. Con una sola semilla, ninguna de las dos diferencias permite afirmar que un modelo supere al otro.

La latencia en la laptop es prácticamente idéntica (unas 37 imágenes por segundo en ambos casos). Esta cifra *no* se puede extrapolar a una OBC: allí cambian el hardware, el motor de inferencia y la precisión numérica.

YOLO26n requirió un 34,9 % más de tiempo de ajuste. Las vistas previas por época suman sólo 10,3 s, por lo que no explican esa diferencia de unos 68 minutos. De todos modos, el costo de entrenamiento se paga una sola vez y fuera de línea, mientras que el costo de inferencia se paga en cada cuadro durante el vuelo.

4.2. Resultados por clase

Clase	AP50–95 YOLO11n	AP50–95 YOLO26n	Δ (p. p.)
pedestrian	12,28 %	12,93 %	+0,65
people	7,14 %	7,32 %	+0,17
bicycle	2,18 %	2,20 %	+0,02
car	47,14 %	46,23 %	–0,92
van	21,92 %	21,53 %	–0,39
truck	17,73 %	17,21 %	–0,52
tricycle	8,96 %	9,59 %	+0,63
awning-tricycle	5,44 %	5,23 %	–0,21
bus	25,28 %	24,09 %	–1,19
motor	12,25 %	12,22 %	–0,03

Tabla 5. AP50–95 por clase sobre validación.

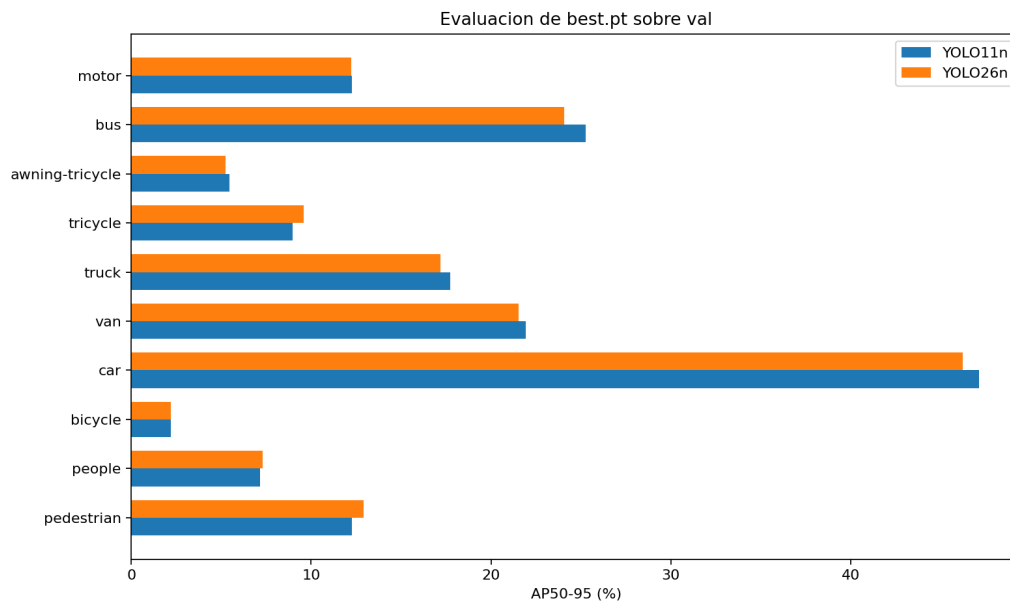


Figura 2. AP50–95 por clase para YOLO11n (azul) y YOLO26n (naranja).

Car es la clase más fuerte en ambos modelos (alrededor del 46–47 %): es un objeto relativamente grande y abundante. Las más débiles son *bicycle* (2,2 %), *awning-tricycle* (5,3 %) y *people* (7,2 %), todas pequeñas o visualmente ambiguas.

YOLO26n mejora levemente *pedestrian* y *tricycle*; YOLO11n obtiene más AP en *bus*, *car* y *truck*. Las mejoras para objetos pequeños que declara el fabricante de YOLO26 no se traducen, en este experimento, en una ganancia clara en las clases pequeñas: *bicycle* mejora 0,02 p. p. y *people*, 0,17 p. p..

4.3. Evolución durante el ajuste

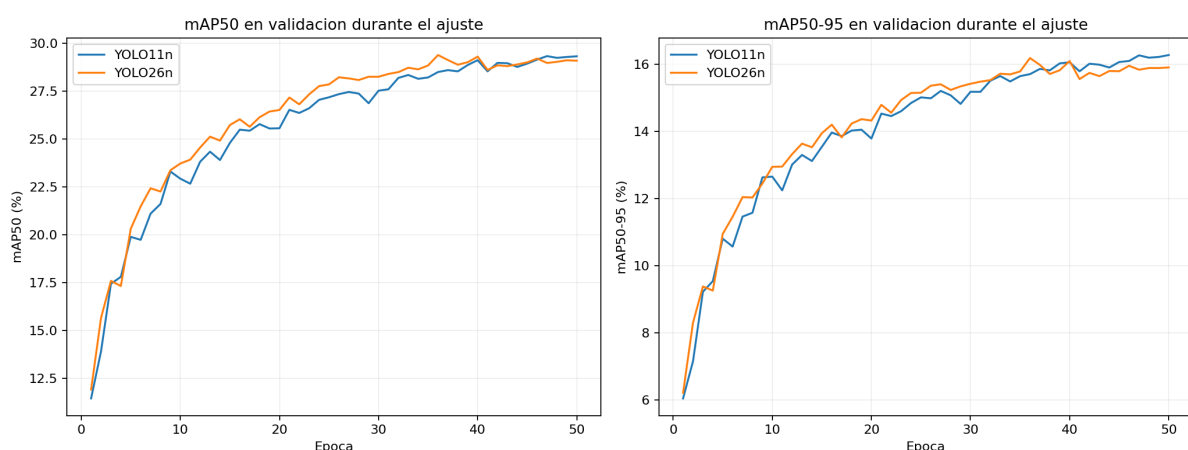


Figura 3. mAP50 y mAP50–95 en validación por época (validación interna de Ultralytics durante el ajuste).

Las curvas crecen con fuerza durante las primeras épocas y se aplanan hacia las épocas 35–40. El máximo de mAP50–95 de YOLO11n aparece en la época 50 (16,28 %). El de YOLO26n aparece en la época 36 (16,19 %) y baja a 15,91 % en la 50, una caída leve que no alcanza para afirmar sobreajuste. Las magnitudes de la pérdida no se comparan entre arquitecturas, porque sus componentes difieren (DFL frente a L1).

Las cifras de la figura 3 provienen de las validaciones internas; la tabla 4 usa la evaluación independiente de best.pt. No son exactamente iguales y la causa no está aislada, por lo que no se mezclan ambos protocolos.

4.4. Análisis cualitativo de errores



Figura 4. Auditoría de errores de YOLO26n (confianza 0,25, IoU 0,5) sobre cuatro de las ocho imágenes de la muestra. Rojo: falsos negativos; cian: falsos positivos.

En la muestra de 871 objetos, YOLO11n obtuvo TP = 263, FP = 192 y FN = 608, y YOLO26n obtuvo TP = 251, FP = 151 y FN = 620. Es decir, YOLO26n comete menos falsas alarmas pero omite algunos objetos más.

La inspección muestra que predominan las omisiones de personas y vehículos de dos ruedas en escenas densas (figura 4), junto con falsas detecciones en vehículos estacionados y vegetación. Se trata de una observación cualitativa sobre escenas relacionadas, no de una muestra representativa. Aun así, coincide con el patrón por clase de la tabla 5: el cuello de botella son los objetos pequeños.

4.5. Discusión y elección del modelo para OBC

Los resultados muestran que **los dos modelos son equivalentes en precisión** para este problema, y que el techo actual lo imponen los objetos pequeños, no la arquitectura. En validación, YOLO11n es descriptivamente más favorable por mAP50-95 y costo de entrenamiento. Sin embargo, para un sistema embarcado importan otros factores:

1. **Posprocesamiento.** YOLO26n no requiere NMS. En escenas densas como las de VisDrone, el NMS procesa muchos candidatos y su costo recae en la CPU de la placa. Un modelo *end-to-end* simplifica la exportación y vuelve más predecible el tiempo por cuadro.
2. **Orientación a borde.** El fabricante diseñó YOLO26 para dispositivos de borde y declara mejoras

de velocidad en CPU. Esa afirmación es exactamente lo que el benchmark en OBC debe verificar.

3. **Costo de entrenamiento.** La diferencia de tiempo de ajuste no influye en la operación del dron.

Por eso se eligió **YOLO26n** como modelo de referencia para el benchmark en placa. Se trata de una decisión fundada en el diseño del modelo, no en una superioridad medida. La sección 7.3 propone incorporar YOLO11n como control para ponerla a prueba.

5. Del laboratorio al dron: requisitos operativos

Las métricas de la sección anterior se obtuvieron sobre imágenes fijas y en una GPU de laptop. Para acercarnos al problema real, conviene traducirlas a magnitudes de la misión: altura de vuelo, campo visual de la cámara, velocidad y cuadros por segundo.

5.1. Tamaño aparente de los objetos

Para una cámara con campo de visión horizontal θ , a una altura h y con apunte cenital, el ancho cubierto en el terreno y la resolución espacial (GSD, *Ground Sample Distance*) son:

$$W = 2 h \tan\left(\frac{\theta}{2}\right), \quad \text{GSD} = \frac{W}{N}, \quad n_{\text{px}} = \frac{L}{\text{GSD}}, \quad (1)$$

donde N es el ancho en píxeles de la imagen que procesa la red y L el tamaño real del objeto.

Ejemplo ilustrativo (supuestos: $h = 50 \text{ m}$, $\theta = 80^\circ$, toma cenital)

$W = 2 \cdot 50 \cdot \tan 40^\circ \approx 83,9 \text{ m}$. Con la imagen nativa ($N = 1400 \text{ px}$), el GSD es de unos $6,0 \text{ cm/px}$; con la entrada de la red ($N = 640 \text{ px}$), de unos $13,1 \text{ cm/px}$. Una persona vista desde arriba ($L \approx 0,5 \text{ m}$) pasa de ocupar unos **8 px** a unos **4 px**, y un auto ($L \approx 4,5 \text{ m}$) pasa de unos **75 px** a unos **34 px**. Este cálculo explica por qué *car* alcanza un AP alto mientras *people* y *bicycle* quedan muy por debajo: al reducir la imagen a 640 px, las personas quedan en el límite de lo que la red puede resolver.

De aquí surgen dos palancas de diseño que deben evaluarse *en la placa*, porque ambas aumentan el costo de cómputo: (a) aumentar la resolución de entrada (por ejemplo, 960 px, que implica alrededor de 2,25 veces más píxeles) y (b) procesar la imagen por mosaicos (*tiling*), como propone SAHI [12]. En ambos casos, además, la altura de vuelo condiciona qué clases es razonable exigir.

5.2. Frecuencia de procesamiento

Si el dron avanza a velocidad v y la OBC procesa f cuadros por segundo, entre dos detecciones consecutivas el dron recorre

$$\Delta d = \frac{v}{f}. \quad (2)$$

Por ejemplo, a $v = 10 \text{ m/s}$, procesar 2 FPS implica 5 m entre detecciones, 5 FPS implica 2 m y 15 FPS implica unos 0,7 m. El umbral aceptable depende de la misión. Un conteo de personas en una zona casi estática tolera pocos FPS; el seguimiento de vehículos en movimiento o la evasión requieren muchos más y una latencia baja y estable. Por eso el benchmark informa, además de la media, los percentiles p50 y p95 de latencia: en control, el peor caso importa tanto como el promedio.

Propuesta

Antes de analizar los resultados de las placas, el equipo debería fijar un **escenario de referencia** (altura, velocidad, cámara y tipo de misión) y derivar de él un **requisito mínimo** de FPS y de latencia p95. Así, el benchmark deja de responder *cuál es más rápido* y pasa a responder *cuál cumple el requisito y con qué margen de precisión, memoria y temperatura*.

6. Siguiente paso: benchmark en computadoras de a bordo

6.1. Preguntas de investigación

- P1.** ¿Cuánto acelera el motor optimizado (TensorRT FP16 o NCNN) respecto de PyTorch *en la misma placa*?
- P2.** ¿Cuánta precisión se pierde al exportar y cambiar la precisión numérica (Δ mAP respecto del .pt en la misma placa)?
- P3.** ¿Se sostiene el rendimiento durante cinco minutos continuos, o aparece limitación térmica (*throttling*)?
- P4.** ¿Qué huella de memoria deja cada variante para el resto del software de a bordo?
- P5.** Con GPU y sin GPU, ¿qué configuración satisface el requisito operativo que defina el equipo?

6.2. Plataformas y variantes

	NVIDIA Jetson (con GPU)	Raspberry Pi 5 (sin GPU de cómputo)
Procesador	SoC ARM64 con GPU integrada CUDA; modelo exacto a registrar por el equipo	SoC ARM64, CPU de 4 núcleos Cortex-A76
Memoria	Compartida entre CPU y GPU	RAM del sistema
Referencia	PyTorch FP32 sobre CUDA	PyTorch FP32 sobre CPU
Optimizado	TensorRT FP16 [9], motor construido en la propia placa	NCNN [11, 10], 4 hilos, Vulkan desactivado
Telemetría	tegrastats, temperaturas, frecuencias	vcgencmd get_throttled, temperaturas, frecuencias
Guía de referencia	Ultralytics para Jetson [7]	Ultralytics para Raspberry Pi [8]

Tabla 6. Configuraciones que se medirán. La comparación principal es *original frente a optimizado dentro de cada placa*. Entre placas se comparan soluciones completas: hardware, motor y precisión numérica diferentes.

6.3. Paquete común y reproducibilidad

Ambos equipos reciben **el mismo ZIP** (unos 61 MiB) con su archivo .sha256. Contiene:

- 300 imágenes únicas de test-dev, elegidas sin reemplazo con semilla 42, junto con sus etiquetas y un orden fijo;
- el best.pt de YOLO26n de la ejecución 20260925T231501Z, con su manifiesto de entrenamiento (**no** debe reemplazarse por pesos descargados de Internet);
- doce imágenes de galería, la configuración y el SHA-256 de cada archivo;
- la evaluación de referencia en la laptop sobre esa misma muestra: **mAP50 = 26,90 %** y **mAP50-95 = 15,19 %**.

6.4. Cadena medida

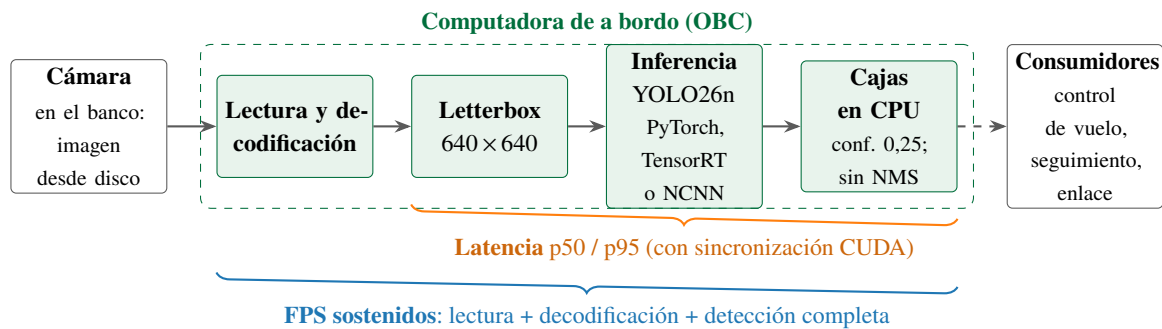


Figura 5. Procesamiento a bordo y los intervalos que mide el benchmark. En vuelo, la cámara reemplaza la lectura desde disco; el dibujo y el guardado de imágenes quedan fuera de la medición.

6.5. Protocolo de medición

- **Repeticiones:** tres corridas de cinco minutos por modelo, cada una con 50 imágenes de calentamiento. El orden alterna entre PyTorch/optimizado, optimizado/PyTorch y PyTorch/optimizado, con pausas de 60 s entre procesos para limitar el sesgo térmico.
- **Aislamiento:** cada medición corre en un proceso nuevo que carga sólo su variante. Nunca se exporta dentro del proceso medido.
- **Entrada:** una imagen por vez desde disco, lote 1, *letterbox* de 640, confianza 0,25 y un máximo de 300 detecciones. No se impone NMS a la salida *end-to-end* de YOLO26.
- **Memoria y telemetría:** RSS antes y después de cargar el modelo, y pico muestreado cada 200 ms, junto con temperaturas, frecuencias y estado de *throttling*. En Jetson la memoria es compartida, por lo que no deben sumarse contadores de CPU y GPU.
- **Condiciones:** misma fuente, refrigeración y modo de potencia para ambos modelos, registrados en cada ejecución. No se modifican `nvpmodel` ni los relojes entre mediciones.
- **Precisión:** tras las seis mediciones, procesos separados calculan mAP50, mAP50–95 y el desglose por clase (confianza 0,001) sobre las 300 imágenes, y generan los paneles anotados.

6.6. Indicadores que se obtendrán

Indicador	Interpretación para el dron
FPS sostenidos (media y dispersión)	Frecuencia real de detección; se contrasta con el requisito $\Delta d = v/f$.
Latencia p50 / p95	Retardo típico y en el peor caso habitual entre la captura y la decisión.
Factor de aceleración	Ganancia del motor optimizado frente al .pt en la misma placa.
Δ mAP frente al .pt	Costo en precisión de exportar y cambiar la precisión numérica.
RSS pico y ahorro de RSS	Memoria disponible para navegación, telemetría y registro.
Temperatura y <i>throttling</i>	Estabilidad durante el vuelo; necesidad de disipación (peso adicional).

Tabla 7. Indicadores del benchmark y su lectura operativa.

6.7. Flujo de trabajo

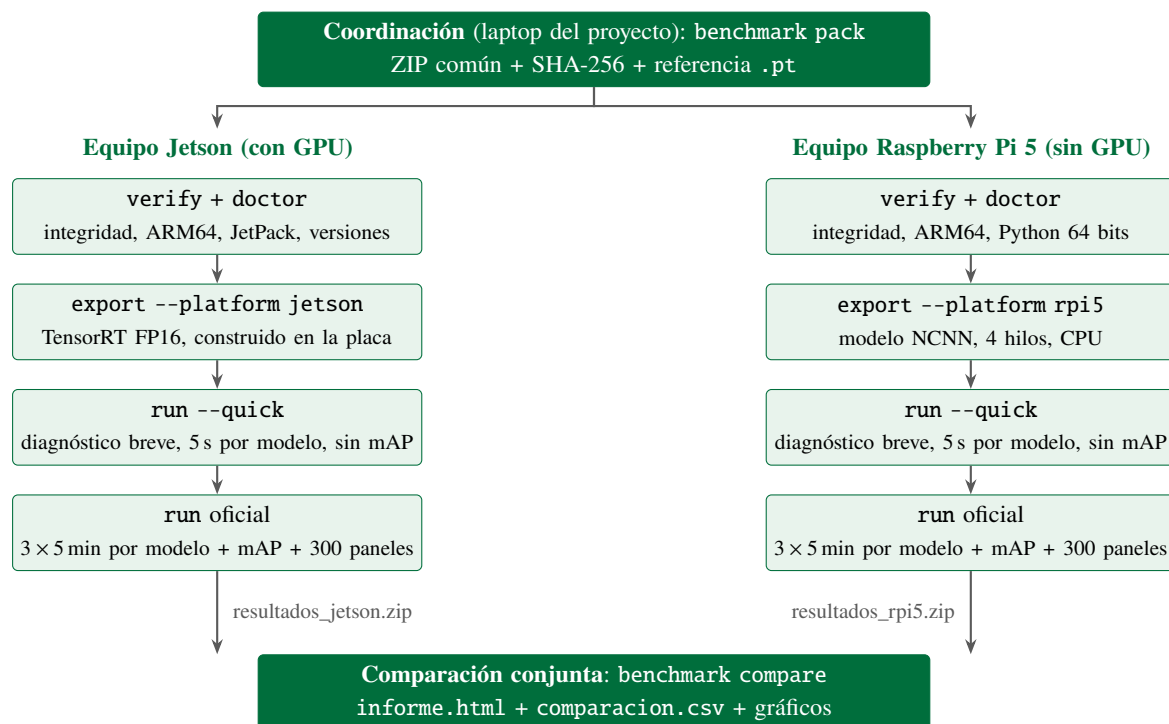


Figura 6. Flujo de trabajo del benchmark (comandos de python `-m tp4.benchmark`). El detalle operativo está en `BENCHMARK_OBC.md`.

6.8. Estado de preparación

Tarea	Estado
Paquete solo YOLO26 generado (package_20260926T172031125947Z) y verificado por CRC, extracción y hashes	Hecho
Selección de las 300 imágenes reproducible con semilla 42	Hecho
Referencia .pt en la laptop (mAP50 26,90 %; mAP50–95 15,19 %)	Hecho
Siete pruebas automatizadas (integridad, rutas, percentiles, rechazos, exclusión de diagnósticos, reporte)	Aprobadas
Inferencia real en CPU y anotación en Windows, marcadas como diagnóstico local	Hecho
Reporte HTML sin resultados abierto sin conexión de red	Hecho
Exportación TensorRT / NCNN y mediciones físicas en las placas	Pendiente

Tabla 8. Validación local realizada el 26 de septiembre de 2026. Ninguna cifra de la laptop se atribuye a las placas.

7. Plan de trabajo

7.1. Tareas, responsables y criterios de aceptación

La tabla 9 ordena el trabajo pendiente. Cada tarea tiene un entregable verificable, de modo que cualquier integrante pueda comprobar si está terminada sin depender de quien la ejecutó.

#	Tarea	Responsable	Criterio de aceptación
1	Publicar el código del benchmark y distribuir el ZIP y su .sha256	Coordinación	Ambos equipos trabajan sobre el mismo <i>commit</i> y el checksum verifica.
2	Preparar el entorno y generar el diagnóstico	Cada equipo	<code>doctor.json</code> con ARM64, Python de 64 bits y versiones; en Jetson, versión de L4T/JetPack.
3	Exportar el modelo optimizado	Cada equipo	El motor se construye en la propia placa y pasa la prueba de carga e inferencia.
4	Ejecutar la prueba corta (<code>--quick</code>)	Cada equipo	Termina con <code>status: finished</code> .
5	Ejecutar el benchmark oficial	Cada equipo	Seis mediciones completas, 300 paneles por modelo, telemetría presente, fuente y refrigeración registradas.
6	Comparación conjunta	Coordinación	<code>informe.html</code> se abre en otra computadora sin red; diagnósticos e intentos incompletos excluidos.
7	Evaluar test-dev completo con el modelo fijado	Coordinación	Evaluación única, sin ajustar hiperparámetros después de verla.
8	Análisis e informe final	Todo el equipo	Discusión del compromiso precisión/velocidad/memoria frente al requisito operativo.

Tabla 9. Plan de trabajo propuesto. Cada ejecución oficial requiere al menos 35 minutos, más exportación, calentamientos y evaluación.

7.2. Reglas para que los resultados sean comparables

- No reentrenar ni modificar checkpoints o hiperparámetros a partir de los resultados del benchmark.
- No instalar ruedas de PyTorch para x86/Windows en la Jetson ni actualizar drivers o JetPack automáticamente. Si no existe una combinación compatible con Ultralytics 8.4.162, informar el bloqueo en lugar de cambiar de motor.
- No reemplazar CUDA por CPU de forma silenciosa. Ante un error, corregir el entorno y lanzar una ejecución nueva, sin mezclar repeticiones de intentos distintos.
- Una diferencia de mAP respecto del .pt debe investigarse como posible efecto de la exportación o de la precisión numérica, no ocultarse.

7.3. Decisiones abiertas y propuestas

Para que el trabajo responda al problema de un dron, y no sólo a un problema de visión artificial, se proponen las siguientes decisiones:

- D1. Asignar hardware y responsables.** Registrar el modelo exacto de Jetson (su generación cambia mucho los resultados), la fuente de alimentación y la refrigeración que se usarían en vuelo.
- D2. Fijar el requisito operativo** (sección 5): escenario de referencia, FPS mínimo y latencia p95 máxima.
- D3. Incorporar YOLO11n como control en placa** (extensión opcional). La elección de YOLO26n se apoya en que no necesita NMS; medir YOLO11n con el mismo protocolo permitiría confirmar si esa ventaja aparece efectivamente en la OBC.
- D4. Medir el consumo energético** (extensión). El protocolo actual no mide energía, y en un dron la batería es el recurso crítico. En Jetson pueden aprovecharse los sensores de potencia que informa

tegrastats; en la Raspberry Pi 5, un medidor USB-C. El indicador sería la energía por inferencia (J/cuadro).

- D5. Explorar resolución y mosaicos** después del benchmark base: entrada de 960 px o SAHI [12] para objetos pequeños, siempre medidos en la placa y frente al requisito.
- D6. Trabajos futuros hacia el sistema completo:** cuantización INT8 con calibración en TensorRT, captura desde cámara real (CSI/USB) en lugar de disco, seguimiento multiobjeto para compensar FPS bajos e integración con el controlador de vuelo.

8. Limitaciones y riesgos

- **Una sola semilla por modelo.** Las diferencias entre YOLO11n y YOLO26n carecen de significancia estadística.
- **Evaluador no oficial.** Las métricas son las de Ultralytics, no las del evaluador oficial de VisDrone, que trata las regiones ignoradas de otra forma.
- **Test-dev parcialmente observado.** Las vistas previas por época de YOLO26n usaron imágenes de test-dev, y el benchmark usa 300 imágenes de ese mismo conjunto. Ninguna decisión de modelo debe basarse en esas observaciones.
- **Muestra de 300 imágenes.** La comparación *relativa* dentro de una misma placa es más robusta que los valores absolutos.
- **FPS desde disco.** Las mediciones no equivalen a los FPS con una cámara real ni incluyen el resto del software de a bordo.
- **Sin medición de energía** en el protocolo base (véase D4).
- **Compatibilidad de software en Jetson.** La combinación de JetPack, PyTorch, TensorRT y Ultralytics 8.4.162 puede bloquear la exportación. Es el principal riesgo de calendario.
- **Discrepancia sin aislar** entre las validaciones internas por época y la evaluación independiente de `best.pt`.

9. Conclusiones

1. El pipeline de datos, entrenamiento y evaluación del TP4 está completo y es trazable: datos auditados con hash, ejecuciones con configuración registrada y artefactos reproducibles.
2. YOLO11n y YOLO26n alcanzan una precisión equivalente sobre VisDrone (mAP50–95 cercano al 16 %). El límite actual lo imponen los objetos pequeños, coherente con la pérdida de resolución al procesar a 640 px.
3. En la GPU de la laptop, ambos modelos procesan unas 37 imágenes por segundo. Esa cifra no es representativa del dron, por lo que la decisión de despliegue debe tomarse con mediciones en la OBC.
4. Se eligió YOLO26n para el benchmark en placa por su salida sin NMS y su orientación a dispositivos de borde. El benchmark en Jetson (con GPU) y Raspberry Pi 5 (sin GPU) ya está diseñado, empaquetado y validado localmente; resta ejecutarlo en el hardware.
5. Para cerrar la brecha entre el laboratorio y el vuelo, el equipo debe fijar un requisito operativo (FPS y latencia) derivado de un escenario de misión y, en lo posible, incorporar la medición de energía.

Referencias

- [1] P. Zhu, L. Wen, D. Du, X. Bian, H. Fan, Q. Hu y H. Ling, “Detection and Tracking Meet Drones Challenge”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, n.º 11, pp. 7380–7399, 2022.
- [2] J. Redmon, S. Divvala, R. Girshick y A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection”, en *Proc. IEEE CVPR*, 2016, pp. 779–788.
- [3] T.-Y. Lin *et al.*, “Microsoft COCO: Common Objects in Context”, en *Proc. ECCV*, 2014, pp. 740–755.
- [4] G. Jocher y J. Qiu, *Ultralytics YOLO11*, 2024. [En línea]. Disponible: <https://github.com/ultralytics/ultralytics>
- [5] Ultralytics, *YOLO26 – Documentación oficial*. [En línea]. Disponible: <https://docs.ultralytics.com/models/yolo26/>
- [6] Ultralytics, *VisDrone Dataset*. [En línea]. Disponible: <https://docs.ultralytics.com/datasets/detect/visdrone/>
- [7] Ultralytics, *Quick Start Guide: NVIDIA Jetson with Ultralytics YOLO*. [En línea]. Disponible: <https://docs.ultralytics.com/guides/nvidia-jetson/>
- [8] Ultralytics, *Quick Start Guide: Raspberry Pi with Ultralytics YOLO*. [En línea]. Disponible: <https://docs.ultralytics.com/guides/raspberry-pi/>
- [9] Ultralytics, *TensorRT Export for YOLO Models*. [En línea]. Disponible: <https://docs.ultralytics.com/integrations/tensorrt/>
- [10] Ultralytics, *NCNN Export for YOLO Models*. [En línea]. Disponible: <https://docs.ultralytics.com/integrations/ncnn/>
- [11] Tencent, *ncnn: high-performance neural network inference framework optimized for the mobile platform*. [En línea]. Disponible: <https://github.com/Tencent/ncnn>
- [12] F. C. Akyon, S. O. Altinuc y A. Temizel, “Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection”, en *Proc. IEEE ICIP*, 2022.

A. Anexo: trazabilidad de artefactos

Artefacto	Ubicación en tp4-yolo/
Ejecución YOLO11n	runs/full/20260925T015117Z/ (run.json, train/results.csv, latency.json, error_sample.json)
Ejecución YOLO26n	runs/full/20260925T231501Z/ (mismos archivos)
Análisis comparativo	analysis/comparacion_yolo11_yolo26.md y figuras
Auditoría de datos	data/audit.json, data/box_review.png
Paquete OBC vigente	benchmark_artifacts/package_20260926T172031125947Z/
Guía operativa OBC	BENCHMARK_OBC.md
Código del benchmark	tp4/benchmark.py, bench_common.py, bench_worker.py, bench_report.py

Tabla 10. Ubicación de la evidencia citada en este informe.

La ejecución 20260925T012307Z figura como *running* pero sólo tiene seis épocas registradas; se excluyó por incompleta. El artefacto epoch_051 de YOLO26n proviene de una llamada adicional al finalizar el entrenamiento y no corresponde a una época entrenada.